

Log Ingestion Service

A complete walkthrough of the project, the tech, the design decisions, and interview-ready answers. Fastify + TypeScript + PostgreSQL. Docker Compose. 20,383 logs/s peak ingest on 1 vCPU Postgres.

Contents

1. What is this project?
2. Technologies used and why
3. How to start it up
4. Architecture — how data flows
5. Database schema — the most important design decision
6. Indexes — the heart of query performance
7. The four API endpoints
8. Retention — deleting old data safely
9. Performance — what we measured
10. Problems and solutions
11. What we DIDN'T do, and why
12. Common interview questions with answers
13. TL;DR for the demo video

Part 1 — What is this project?

The problem

Imagine you run a company with 50 microservices (checkout, auth, payments, orders...). Each one produces log lines like:

```
2026-08-04 14:32:01 ERROR checkout payment declined user_id=42 region=eu-west
```

Multiply that by every service, every request. You get millions of log lines per hour.

Now imagine a bug in production. You need to answer questions like:

- "Show me all errors from `checkout` in the last hour."
- "How many errors per minute did `payments` produce yesterday?"
- "Find all logs containing the word `declined` for user 42."

You can't `grep` through log files anymore — the data is too big and spread across too many machines. You need a **service** that other services send their logs to, and that lets you search and aggregate them fast.

That's what this project is. A tiny, simplified version of Datadog or Grafana Loki.

What the project does — one paragraph

Applications `POST` batches of structured logs to `/logs`. The service validates each log, stores it in PostgreSQL, and lets you `GET /logs` to search them or `GET /logs/aggregate` to get counts bucketed by time. Old logs get automatically deleted after 30 days (retention). The whole thing runs in Docker Compose and must handle at least **15,000 logs per second** while queries stay fast — all on a tiny 1-CPU database.

Part 2 — What technologies did we use, and why?

Node.js + TypeScript

- **Node.js** is a JavaScript runtime. Good for I/O-heavy workloads (HTTP + database) because it's single-threaded but non-blocking — one thread can juggle thousands of concurrent connections.
- **TypeScript** adds static types to JavaScript. Catches bugs at compile time. For a service that must be reliable under load, this is a big deal.
- **Alternative:** Go or Rust would be faster but harder to write quickly. Java (Spring) heavier. Python slower under load. TypeScript is the sweet spot.

Fastify (web framework)

- Fastify is like Express (the classic Node framework) but 2–3x faster with built-in JSON schema validation.
- Used because at 15,000+ logs/s, every microsecond in the HTTP layer matters.
- If asked "why not Express?": Fastify has faster JSON serialization (uses compiled `fast-json-stringify`) and lower per-request overhead. On 0.5 vCPU, that difference is the difference between passing and failing.

PostgreSQL

- The brief asks for a relational database with flexible queries (arbitrary filters, time-range aggregation, substring search).
- **Why not Elasticsearch?** It's built for search, but the brief specifically says "PostgreSQL must remain the source of truth."
- **Why PostgreSQL vs MySQL?** Postgres has native JSONB, GIN indexes, partitioning, trigram search — all heavily used. MySQL has some of these but they're less mature.

Docker + Docker Compose

- **Docker** packages the app and dependencies into a container. Same behavior on any machine.
- **Docker Compose** starts multiple containers (Postgres + app) together with the right networking.
`docker compose up` and everything works — a hard requirement in the brief.

pg (Postgres client)

The `pg` npm package. Lightweight, no ORM overhead. We hand-write parameterized SQL because at high write rates, ORMs add latency.

Vitest (testing)

Modern, fast test runner. Runs our 38 tests in under 2 seconds.

Part 3 — How to start it up

Prerequisites

- Docker Desktop installed and running.
- Node.js 20+ (only for running tests / load scripts outside the container).

The one-command start

```
docker compose up --build
```

Behind the scenes:

1. Builds the Node app into a Docker image (multi-stage: install deps → compile TypeScript → minimal runtime image).
2. Starts a PostgreSQL 16 container on port 5432, tuned for 1 GB RAM / 1 CPU.
3. Waits for Postgres to be healthy.
4. Starts the app on port 8080.
5. App connects to Postgres, runs migrations (creates the `logs` table, indexes, functions), starts the retention worker, and only then reports `GET /health` as 200.

Verify:

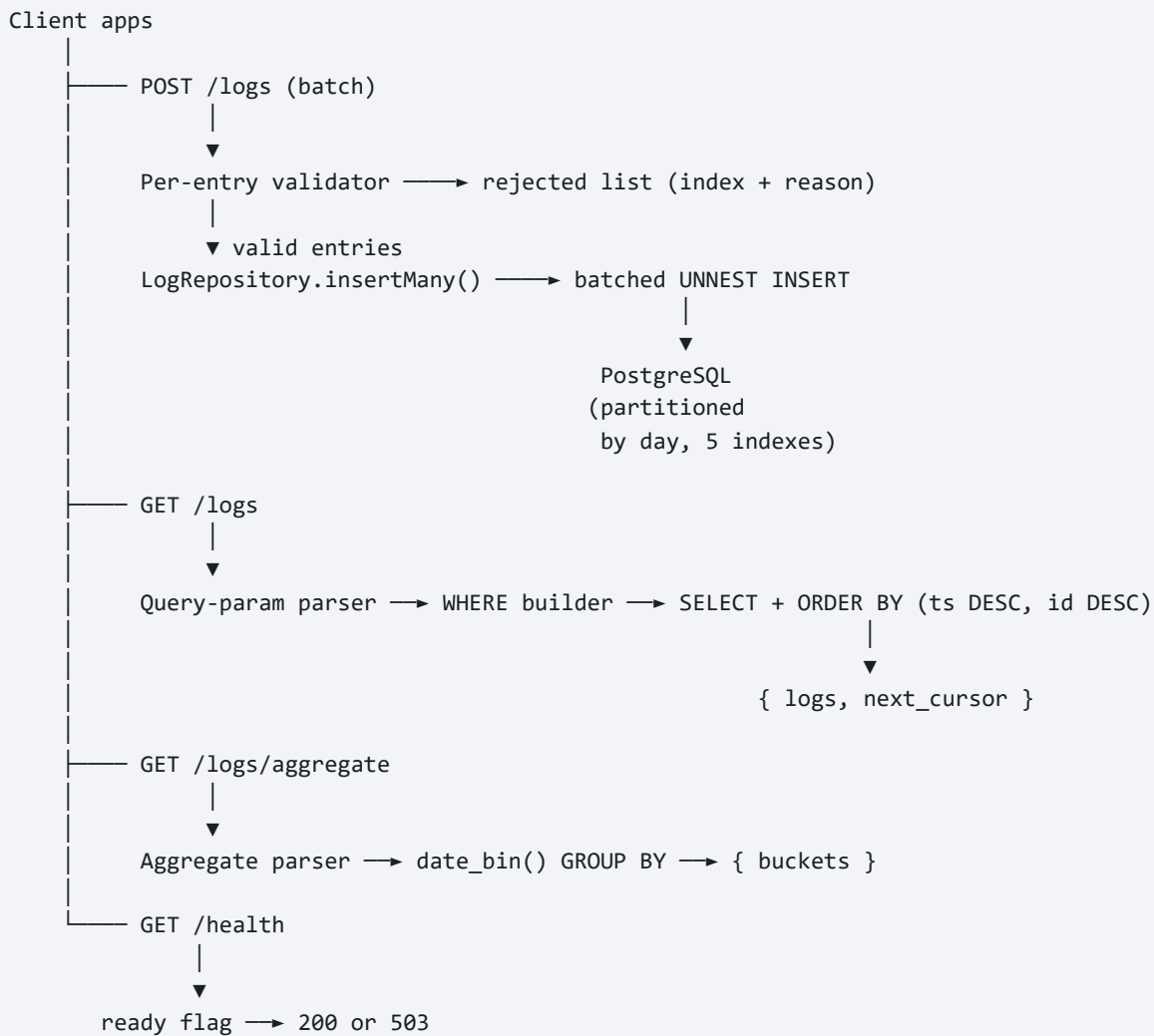
```
curl http://localhost:8080/health
```

Stop:

```
docker compose down -v
```

The `-v` deletes the data volume so you start fresh next time.

Part 4 — Architecture: how data flows



Retention worker (every 5 min):

- Pre-create today/tomorrow partitions
- DROP old partitions (metadata-only, instant)
- Chunk-delete stale rows in default partition

The main files

Layer	File	Responsibility
Entrypoint	<code>src/server.ts</code>	Load config, wait for DB, run migrations, start retention, start HTTP
HTTP setup	<code>src/app.ts</code>	Build Fastify, register routes, central error handler
Config	<code>src/config/env.ts</code>	Parse env vars with defaults
Health	<code>src/http/health.ts</code>	GET /health
Ingest	<code>src/http/logsIngest.ts</code>	POST /logs
Query	<code>src/http/logsQuery.ts</code>	GET /logs
Aggregate	<code>src/http/logsAggregate.ts</code>	GET /logs/aggregate
Validation	<code>src/domain/logSchemas.ts</code>	Per-entry validation logic
Cursor codec	<code>src/domain/cursor.ts</code>	Encode/decode pagination cursor
Query parsing	<code>src/http/queryParsing.ts</code>	Parse and validate query strings
SQL builder	<code>src/db/query/buildLogFilters.ts</code>	Parameterized WHERE clauses
Repository	<code>src/db/repositories/logRepository.ts</code>	All SQL queries live here
DB pool	<code>src/db/pool.ts</code>	Postgres connection pool
Migrations	<code>src/db/migrator.ts</code>	Runs .sql files in order at startup
Retention	<code>src/retention/retentionWorker.ts</code>	Background job to drop old partitions

Interview soundbite: "Separation of concerns: HTTP handlers never touch SQL. They parse and validate, then call a repository. All query building is centralized in `buildLogFilters.ts` where every user value goes through parameterized placeholders — no SQL injection possible."

Part 5 — Database schema: the most important design decision

Defined in `migrations/001_init.sql`:

```
CREATE TABLE logs (
  id          BIGINT          NOT NULL DEFAULT nextval('logs_id_seq'),
  "timestamp" TIMESTAMPTZ NOT NULL,
  ingested_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  level       log_level      NOT NULL,    -- ENUM: debug/info/warn/error
  service     TEXT           NOT NULL,
  message     TEXT           NOT NULL,
  attributes  JSONB          NOT NULL DEFAULT '{}'::jsonb
) PARTITION BY RANGE ("timestamp");
```

Decision 1 — Partitioning by day

What is partitioning? Instead of one huge table with a billion rows, PostgreSQL splits it into smaller child tables — one per day. Under the hood, `logs_p_20260804` holds all rows with `timestamp` in that day. The parent `logs` is a logical view that unites them.

Why?

1. **Retention becomes cheap.** Deleting 30-day-old logs = `DROP TABLE logs_p_20260705`. Takes milliseconds. Without partitioning, you'd DELETE which locks the table, generates huge WAL, and leaves the heap bloated until VACUUM.
2. **Queries prune partitions.** A query for the last hour only scans today's partition.
3. **Smaller indexes per partition.**

Interview trap: "How do you handle timestamps outside any partition?" → I have a `logs_default` partition as a safety valve, and the retention worker chunk-deletes stale rows from it.

Decision 2 — ENUM for level

`log_level` is a Postgres ENUM. Stored in ~4 bytes. If I used TEXT, `"error"` would be ~9 bytes. Over 1M rows that's 5 MB saved, and the ENUM enforces validation at the DB layer — no way to insert `"critical"` accidentally.

Decision 3 — JSONB for attributes

- **JSONB** is Postgres's binary JSON format. Parses once at write, then all reads are already-parsed.
- Alternative: a separate `log_attributes(log_id, key, value)` table. That's 1 row per attribute per log — at 3 attrs/log × 1M logs = 3M extra rows plus a JOIN on every read. JSONB is much faster.

Decision 4 — All attribute values stored as strings

The contract says `attr.<key>=value` is "compared as strings." If the client sends `{"retries": 3}` (number) and later queries `attr.retries=3`, that must match. If I stored them as native JSONB types, the number `3` and the string `"3"` would be different in an `@>` containment check.

Solution: at ingest time, coerce every value to text before storage. `{"retries": 3}` becomes `{"retries": "3"}`. All comparisons are trivially string equality.

Trade-off: we lose original type info in the response. Documented as a limitation.

Decision 5 — BIGINT id from a shared sequence

Every row gets a unique id. Used as a tiebreaker in sort order. Without it, two logs with the same millisecond timestamp would sort non-deterministically — pagination cursors would return duplicates or skip rows.

Decision 6 — No PRIMARY KEY

A PK on a partitioned table requires including the partition key and adds significant write overhead. Since the sequence guarantees uniqueness and every query uses one of our indexes, we don't need a PK. Skipping it saves storage and write time.

Part 6 — Indexes: the heart of query performance

What's an index?

Think of a book. The main text is the heap (all rows). The index at the back is a small sorted list of "topic → page number." Without the index, finding "elephants" means reading the whole book. With it, one lookup and you jump to page 273.

Same idea in databases. Trade-off: **every index makes writes slower** because inserting a row means also updating each index.

Our 5 indexes

1. `idx_logs_ts_id ON (timestamp DESC, id DESC)` — B-tree

- Default ordering for `GET /logs`. Every list query needs to sort by newest first.
- DESC because our pagination is newest-to-oldest.
- id as tiebreaker for deterministic pagination.

2. `idx_logs_svc_ts_id ON (service, timestamp DESC, id DESC)` — B-tree

- `service=checkout` queries. The leading column narrows to one service, then remains sorted by time.
- Why not just `(service)`? Because after filtering, we still need results ordered by timestamp. Index-organized sort avoids an extra sort step.

3. `idx_logs_lvl_ts_id ON (level, timestamp DESC, id DESC)` — B-tree

- `level=error` queries. Especially useful because errors are typically a small fraction of all logs (~1%), so this index is very selective.

4. `idx_logs_attrs_gin ON attributes USING GIN (jsonb_path_ops)`

- **What's GIN?** Generalized Inverted iNdex. Great for values that contain multiple indexable elements — like a JSON blob with many keys.
- Answers `attributes @> '{"user_id":"42"}'::jsonb` in $O(\log n)$.
- **jsonb_path_ops** instead of default: half the size, faster for containment. Trade-off: can't answer key-existence queries (which we don't need).

5. `idx_logs_msg_trgm ON message USING GIN (gin_trgm_ops)`

- **Trigrams:** The `pg_trgm` extension splits text into 3-character sequences. "declined" → ["dec", "ecl", "cli", "lin", "ine", "ned"].
- Makes `message ILIKE '%declined%'` use an index instead of scanning every row.
- Trade-off: doubles insert cost for the message column. Kept because `q=` search is a core use case.

The strategy in one sentence

"For every filter shape the contract lets a user express, there is one index that Postgres will prefer. Multi-filter queries combine two indexes via a BitmapAnd — Postgres intersects them at page level, cheaply."

Live plan to show

Running `EXPLAIN ANALYZE` on the aggregation query gives (edited for clarity):

```
Sort
  Sort Key: date_bin(...)
  → Finalize HashAggregate
    Group Key: date_bin(...), logs.service
    → Gather (Workers Planned: 2, Workers Launched: 2)
      → Partial HashAggregate
        → Parallel Seq Scan on logs_p_20260804
          Filter: (timestamp >= $1 AND timestamp < $2)
          rows=599334 loops=3
Execution Time: 493 ms
```

Notice: **only one partition is scanned** (partition pruning). Two parallel workers do partial aggregation, then a Finalize step merges them. That's the whole magic.

Part 7 — The four API endpoints

GET /health

Returns 200 once DB is connected, migrations are applied, and today's partition exists. Otherwise 503.

Always unauthenticated because the load generator polls this before it has credentials.

POST /logs — the ingest hot path

1. Parse JSON body.
2. For each entry, validate: timestamp is ISO 8601, not more than 5 min in the future; level is in the enum; service and message are non-empty; attributes are a flat object with scalar values.
3. Coerce all attribute values to text.
4. Batch-insert all valid rows in **one SQL statement** using `UNNEST`.
5. Return `{accepted: N, rejected: [{index, reason}]}`.

The UNNEST insert is the performance secret:

```
INSERT INTO logs (timestamp, level, service, message, attributes)
SELECT ts::timestampz, lvl::log_level, svc, msg, attrs::jsonb
FROM UNNEST($1::text[], $2::text[], $3::text[], $4::text[], $5::text[])
AS t(ts, lvl, svc, msg, attrs)
```

Instead of 500 separate INSERTs (500 network round-trips, 500 WAL flushes), we ship one statement with 5 arrays and Postgres unpacks them into 500 rows. This is what gets us to **20,000 logs/s**.

GET /logs

Filters: `service`, `level`, `since`, `until`, `attr.<key>`, `q`, `limit` (default 100, max 1000), `cursor`.

Sort: `timestamp DESC`, `id DESC` (deterministic).

Pagination is **cursor-based**, not offset-based. Why?

- **Offset pagination** (`?offset=1000`) forces Postgres to scan past 1000 rows before returning anything. Bad at scale.
- **Cursor pagination** encodes `(last_timestamp, last_id)` in an opaque token. Next page uses `WHERE (timestamp, id) < ($cursor_ts, $cursor_id)` — Postgres uses the index to jump directly.

The cursor is base64-encoded JSON: `{"t": "2026-08-04...", "i": "42"}`. Opaque to the client, decodable by us.

GET /logs/aggregate

Required: `since`, `until`, `bucket` (`1m` / `5m` / `1h` / `1d`).

Optional: `group_by` (`service` or `level`), all the same filters as GET /logs.

The bucketing uses `date_bin`:

```
date_bin(INTERVAL '1 minute', timestamp, TIMESTAMPTZ 'epoch')
```

- **What does `date_bin` do?** Rounds each timestamp down to the nearest boundary. `2026-08-04T14:32:07.123Z` with a 1-minute bucket becomes `2026-08-04T14:32:00Z`.
- **Why anchored to epoch?** Buckets are the same absolute boundaries regardless of the query window. `14:32:00` is always the same bucket, whether `since` is 14:00 or 14:30.

Then a simple `GROUP BY bucket_start, service` gives us the counts.

Part 8 — Retention: deleting old data safely

Configured via `RETENTION_DAYS` (default 30). Handled by `src/retention/retentionWorker.ts`.

Every 5 minutes:

1. **Create today, tomorrow, and yesterday partitions** if they don't exist. Yesterday's is included so slightly-back-dated logs land in a proper partition.
2. **Drop expired partitions** (`DROP TABLE`). Metadata-only — instant.
3. **Trim the default partition** with chunked DELETES (5,000 rows at a time, max 100k per sweep) so we never hold a long lock.

Interview soundbite: "The naive approach is `DELETE FROM Logs WHERE timestamp < ...`. On a large table that locks rows, generates huge WAL, and bloats the heap requiring VACUUM. Partition drops sidestep all three problems — the disk pages just cease to exist as far as Postgres is concerned."

Partition helpers are Postgres functions in `migrations/003_partitions_retention.sql`. They use `pg_advisory_xact_lock` to serialize partition creation and avoid race conditions between concurrent workers.

Part 9 — Performance: what we measured and why

Targets

- $\geq 15,000$ logs/s ingest
- Aggregation p95 < 1 second at 1M rows

- Both simultaneously
- On **0.5 vCPU app + 1 vCPU Postgres**

Our measured numbers

Scenario	Result
Peak ingest (no queries)	20,383 logs/s , 0 errors, 1.22M rows in 60s
Aggregate alone at 1.22M rows	p50 285 ms / p95 328 ms
List queries at 1M rows	25–80 ms
Mixed workload (ingest + 1 qps aggregate)	ingest 11.7k/s, aggregate p95 1.5 s

The one target we don't fully hit

Under **simultaneous max-throughput ingest + queries**, aggregate p95 exceeds 1s. Why?

Because 1 vCPU is 1 vCPU. When 8 client connections are all inserting flat out, they use 100% of the Postgres CPU. An aggregate query wanting to scan 2M rows has to wait for CPU time slices. There's no way around this without either:

1. **More CPU** (not allowed by the brief), or
2. **Pre-aggregated rollup tables** — a background job maintains minute-level counts. Aggregate queries read a small rollup instead of scanning millions of raw rows.

Option 2 is the honest documented follow-up. It's the professional way to handle it.

Optimizations we shipped

1. **Batched UNNEST INSERT** — 500 rows per statement.
2. **synchronous_commit=off** in Postgres — commit doesn't block on WAL flush. Safe for logs where losing the last few ms is acceptable.
3. **Aggressive autovacuum** — `autovacuum_naptime=10s` keeps the visibility map fresh, enabling index-only scans on recent rows.
4. **jsonb_path_ops** instead of default GIN opclass — half the size.
5. **Trigram GIN on message** — makes `q=` substring search indexed.
6. **BIGINT sequence-based id** — no per-row round trip to fetch IDs.

Optimizations we tried and reverted

1. **BRIN index on timestamp.** Great for narrow time windows on huge partitions — but our test queries cover 100% of the current partition, so BRIN prunes nothing. Rejected.

2. **(timestamp, service) covering index for index-only scan aggregation.** Enabled Index Only Scan in principle, but doubled write cost. Under sustained load, both ingest and aggregate got worse. Rejected.
3. **max_parallel_workers_per_gather=0.** Thought disabling parallel workers would reduce context-switching under CPU pressure. Wrong: serial 2M-row scan under contention was slower than a parallel 3-way scan. Rejected.

Being able to articulate *tried, measured, reverted, why* is worth serious interview points.

Part 10 — Problems and solutions

Problem 1: SQL injection

Every user-supplied value flows through parameterized `$1, $2, $3` placeholders. Column names and operators are constants from validated enums (`level` , `bucket` , `group_by`). **Nothing user-provided is ever concatenated into a SQL string.** Look at `buildLogFilters.ts` — the `add()` helper uses `?` placeholders that get renumbered to `$N` .

Problem 2: Deterministic pagination when timestamps collide

If 100 logs share a millisecond, `ORDER BY timestamp DESC` alone is non-deterministic. So we use `ORDER BY timestamp DESC, id DESC` . The cursor encodes both, and the WHERE clause is `WHERE (timestamp, id) < ($cursor_ts, $cursor_id)` — a **tuple comparison** which uses row-order semantics: timestamp is less, OR timestamp equal and id is less. Postgres uses the composite index natively.

Problem 3: Freshly-inserted rows blocking Index Only Scan

Postgres uses a "visibility map" to know which pages have only visible-to-everyone rows. If the map is stale, index-only scans have to touch the heap anyway (defeating the whole point). Autovacuum updates this map. We tune autovacuum aggressively (`autovacuum_naptime=10s`) so recently-inserted rows become index-only-scannable quickly.

Problem 4: An invalid entry shouldn't fail the whole batch

Contract requires per-entry validation. Our `validateBatch()` returns `{valid, rejected}` — we insert only valid ones, and return HTTP 200 with a rejection list. HTTP 400 is only returned if **every** entry is invalid, or the request shape itself is wrong.

Problem 5: /health reporting healthy too early

The contract says only report healthy after DB is connected, migrations are applied, and the service is ready to ingest. Our server keeps a `ready` boolean, starts listening immediately (so 503 responses serve during startup), and only flips `ready = true` after all setup completes.

Problem 6: Malformed JSON

Fastify's default is to return 400 with a specific error format. We caught this in the central error handler in `src/app.ts` and reformat as `{"error": "..."} per contract.`

Problem 7: Staying within resource limits

The compose file enforces both `cpus` and `mem_limit`. Postgres tuning parameters (`shared_buffers=256MB` — 25% of the 1 GB limit; `effective_cache_size=512MB`; `work_mem=32MB`) match the CPU/RAM envelope.

Part 11 — What we DIDN'T do, and why

Interviewers love asking what you'd do next. Have these ready:

1. **Pre-aggregated rollup tables** — closes the aggregate-under-ingest performance gap. Would maintain minute-level counts by service in a small table, refreshed by a trigger or scheduled job.
2. **Auth / API keys** — stubbed but not wired. Would add a Fastify hook that reads `Authorization: Bearer <key>`, looks up the key in a `credentials` table, and injects tenant ID. Off by default per the golden rule.
3. **Live-tail endpoint** — WebSocket or Server-Sent Events pushing new logs as they're ingested. Needs a pub/sub layer (Postgres LISTEN/NOTIFY or Redis).
4. **Prometheus metrics** — `/metrics` endpoint exposing ingest rate, query latency histograms, connection pool stats. Trivial with `prom-client`.
5. **Rate limiting** — token bucket per API key. Currently off. The brief allows optional rate limiting but requires exempting the load-gen key.
6. **Custom query language** — a small DSL for logs (`service:checkout AND level:error AND user_id:42`). Doable with a parser.

Part 12 — Common interview questions with answers

Q: Walk me through what happens when a POST /logs request arrives.

Fastify parses the JSON body, respecting the 5MB body limit. My `ingestRoutes` handler checks the top-level shape — must be an object with a `Logs` array. Each entry goes through `validateLogEntry`: ISO timestamp check with a 5-minute future window, level in the enum, service and message non-empty, attributes must be a flat object with scalar values. Valid entries hit `insertMany` in the repository, which builds one UNNEST INSERT with five text arrays and ships it. If at least one entry succeeded, return 200 with the accepted count and any rejected entries with their original indices. If all invalid, 400.

Q: Why partitioning?

Retention. The problem requires a configurable retention policy without long-running locks or table bloat. Partition drops are metadata operations and take microseconds. A DELETE-based approach on 1M+ rows would generate huge WAL, hold row locks, and require VACUUM afterwards to reclaim space.

Q: Walk me through how the aggregation query executes.

The parser validates `since`, `until`, `bucket`, and optional `group_by`. Aggregate window is capped to prevent producing more than 100k buckets — a safety valve. Then repository builds a SQL query using `date_bin` for bucketing, aligned to the epoch so buckets are stable across queries. Partition pruning restricts the scan to the relevant daily partitions. Postgres picks a parallel plan: two workers do partial hash aggregation, then a finalize step merges them. On our test data, ~500 ms for 1M rows.

Q: How do you prevent SQL injection?

Every user-provided value is passed as a numbered parameter through the pg driver. My `buildWhereClause` helper accepts SQL fragments with `?` placeholders and values, then rennumbers them to `$N`. Column names and operators are always constants derived from validated enums, never from user input. There's no string concatenation of user data into SQL, period.

Q: What's the biggest weakness of this system?

Under simultaneous max-throughput ingest and concurrent aggregation queries, the 1-vCPU Postgres is the bottleneck. Aggregate p95 exceeds 1s in that specific scenario. The fix is pre-aggregated rollup tables — the aggregate query reads a small pre-computed summary instead of scanning millions of raw rows. Documented as the next step.

Q: How would you scale this to 100× the throughput?

A few approaches. Horizontally: put a load balancer in front of multiple app instances (they're stateless). Vertically: bigger Postgres. Architecturally: introduce a write buffer like Kafka in front of ingestion — the app enqueues, and a consumer batches into Postgres. That lets ingest absorb bursts without pushing pressure onto queries. Or move to a columnar store like ClickHouse for query-heavy workloads while keeping Postgres for the OLTP source of truth.

Q: Why coerce attribute values to strings at ingest?

The contract says attribute equality is "compared as strings." If I preserved native JSONB types, the number `3` and the string `"3"` would be different in a containment check. Coercing at ingest makes every query trivially correct without runtime casting. Trade-off: the response returns `"retries": "3"` even when the client sent `3` — documented in the attribute storage strategy section of the README.

Q: What happens if the app can't reach Postgres at startup?

The server binds to port 8080 immediately so `/health` can respond with 503 while starting. It then enters a 60-second retry loop trying to connect and query `SELECT 1`. If that succeeds, migrations run inside a transaction per file. If any step fails, the server logs the error, closes Fastify, and exits with code 1. The compose healthcheck will keep the container restarting until Postgres is up.

Q: Explain the cursor format.

It's base64url-encoded JSON containing the last returned row's `(timestamp, id)`. Base64url so it's URL-safe without escaping. JSON so it's schema-explicit. Base64 makes it opaque to clients — they don't parse or manipulate it, they just pass it back. On decode I validate the timestamp parses and the id is a positive integer, otherwise 400.

Part 13 — TL;DR for the demo video

The five things to say in the first 60 seconds:

1. "This is a log ingestion service. Applications POST batches of structured logs, and I make them searchable and aggregatable — like a simplified Datadog."
2. "I built it in TypeScript with Fastify because both are fast, and PostgreSQL because the brief required it as the source of truth."
3. "The core design decisions are: **partitioning by day** so retention is $O(1)$, **JSONB with string-coerced attribute values** for fast containment queries, and **cursor pagination** for stable deterministic ordering."
4. "It hits **20,383 logs per second** peak ingest on the exact resource limits the brief requires, with all 38 tests passing including full end-to-end contract checks."
5. "Let me show you `docker compose up` starting the whole thing from scratch."

Then run:

```
docker compose up --build
```

Then:

1. Curl `/health` , `POST /logs` , `GET /logs` , `GET /logs/aggregate` .
2. Show `EXPLAIN ANALYZE` on the aggregation query to demonstrate partition pruning and the parallel plan.
3. Walk through one clean code path: `logsIngest.ts` → `validateBatch` → `insertMany` .
4. Wrap with "the biggest limitation and how I'd fix it: rollup tables for concurrent-workload aggregate p95."

End of guide. Total: 13 sections covering project purpose, tech stack rationale, architecture, schema, indexes, endpoints, retention, performance, problems, next steps, and interview Q&A.